

Where Structure Helps: RDF vs. Property Graphs for Explainable Legal GraphRAG

Vishnu SANKARAKUMAR ^{a,1} and Anelia KURTEVA ^{a,2}

^a*School of Computer Science, University of Birmingham, UK*

Abstract. Large Language Models (LLMs) are increasingly explored as advisory tools for navigating law. However, their tendency to hallucinate and not fully understand complex contexts makes them unsafe, as incorrect or incomplete legal guidance can carry severe consequences. Recent efforts have shifted toward integrating knowledge graphs (KGs) and LLMs via the Graph Retrieval Augmented Generation (GraphRAG) approach. By providing a structured data infrastructure, KGs offer a mechanism to constrain LLM outputs and ensure they remain grounded in verifiable information. In such an approach, data can be represented as a labelled property graph (LPG) or as an RDF KG. To our knowledge, no prior work directly compares the two for retrieval quality in GraphRAG for legal question answering. We present such a comparison over the EU AI Act. We propose a custom pipeline for reverse-engineering an automatically generated LPG and extracting an RDF ontology with zero data loss. GraphRAG systems are then implemented with both the LPG and the RDF KG backends. Both systems are evaluated on 50 AI Act-related questions across five complexity tiers using RAGAS, scoring faithfulness, context precision, and context recall. The result is a complexity-dependent crossover. The LPG-based GraphRAG is more faithful at every tier and stronger overall on simple factual questions. In comparison, the RDF-based system achieves higher context recall on the three hardest tiers, yielding 0.76 against 0.59 on the analytical tier. Structure is not uniformly better: the LPG’s schema-free retrieval wins on simple questions, and the ontology’s breadth only pays off once questions require traversal. In most legal compliance checks and guidance-seeking, the hard questions are the ones that matter. On those questions, the RDF KG recovers more of the needed information while keeping retrieval explainable.

Keywords. GraphRAG, Labelled Property Graph, RDF Knowledge Graph, Explainable Retrieval, Provenance, Auditability

1. Introduction

Large Language Models are increasingly being explored as advisory tools for navigating complex regulation, but their core limitation of hallucination makes them unsuitable for safety-critical domains without additional safeguards. A professional needing concrete advice about their AI system cannot tolerate a confidently wrong answer because there can be legal and financial consequences to bad guidance. The technology is not trustworthy enough to deploy without concrete domain knowledge.

¹E-mail: vxs268@student.bham.ac.uk.

²Corresponding Author: Anelia Kurteva, School of Computer Science, University of Birmingham, UK; E-mail: a.kurteva@bham.ac.uk.

Retrieval-Augmented Generation (RAG) addresses this by taking relevant context from a separate knowledge base and passing it to the LLM with the user’s query, to ensure the response is grounded in source material rather than the model’s pre-training. In standard RAG, the knowledge base is a collection of text chunks indexed by vector embeddings. GraphRAG replaces the flat data store with a structured knowledge graph made up of entities and relationships, enabling the system to traverse connections between concepts rather than relying on semantic similarity. This is important for a domain like the EU AI Act [1] where relevant information spans a dense and highly cross-referenced document containing hundreds of recitals and articles. In practice, a compliance auditor, lawyer, or student queries a natural language interface about the EU AI Act. They ask a question, the system retrieves relevant context from the knowledge graph, and an LLM generates an answer grounded in specific domain knowledge.

Knowledge graphs can be implemented in two main ways: the first is a Labelled Property Graph (LPG) storing entities as nodes connected by directed and typed relationships that can store properties on edges. Neo4j [7] is the standard LPG database, queried using the Cypher language. The second is the Resource Description Framework (RDF) [3], which stores knowledge as subject-predicate-object triples with globally unique URIs, and is supplemented with the Web Ontology Language (OWL) [4] to declare formal class hierarchies, relationship rules and constraints. The main query language for RDF graphs is SPARQL [5]. Both implementations have different strengths.

Motivated by this, our paper aims to answer the following research question: how does the choice of knowledge graph representation affect both retrieval quality and the explainability of regulatory question answering as questions grow more complex? To answer this, we make three contributions. First, an empirical one: across 50 questions in five complexity tiers, LPG and RDF retrieval cross over, with the property graph stronger on simple lookups and the ontology stronger on the complex, multi-provision questions that matter for compliance. Second, an explanatory one: we show why this happens, discussing the inspectable paths of ontology structure and the leaner retrieval of the schema-less graph. Third, an architectural one: we argue for a complexity-dependent system that selectively applies context post-processing. We make the direct comparison forming the foundation of this paper possible by developing a transformation pipeline that converts an LPG into a formally structured RDF ontology with no data loss.

2. Background

2.1. LPG vs RDF for GraphRAG

An LPG stores entities as nodes joined by typed, directed edges that can carry properties, queried with Cypher. It has no formal schema: labels are free strings. RDF stores knowledge as subject-predicate-object triples with global URIs, and OWL adds a formal layer of class and property hierarchies on top, queried with SPARQL. The distinction that matters for retrieval is that in RDF the relationships between types, and between predicates, are themselves declared and queryable, whereas in an LPG they are not. Section 4.1.2 shows why this drives the results.

2.2. Retrieval Approaches in Literature

The retrieval component of a RAG system determines how context is selected from the knowledge graph before being passed to the LLM. Four existing retrieval architectures informed project design.

PharmaBot [12] combines semantic and structural (Node2Vec [13]) embeddings, which motivated using vector similarity as the first retrieval step in both pipelines, rather than relying on query translation alone. PolyG [14] matches query type to question complexity, handling any question at the cost of latency. Iterative retrieval [15] re-ranks context over several rounds to cut noise: the most capable multi-hop approach, and the most expensive. A graph-enhanced e-commerce system [16] runs vector and Cypher retrieval in parallel. While these papers are concerned with highly relevant ideas, none of these directly compares an LPG against an RDF backend on the same data for retrieval quality. These papers and related existing LPG-versus-RDF comparisons measure storage and query performance, not answer generation. This paper fills that gap.

The ideal pipeline would involve similarity localisation as first step, graph traversal matched to question complexity, and iterative context re-ranking to cut noise. In the course of development, question type classification and context re-ranking were dropped in favour of a more streamlined approach to mitigate scope drift. Despite that, the evaluation results empirically validate the design principles behind those existing techniques that were not implemented.

2.3. Semantic Representations of the AI Act

A parallel line of work encodes regulation itself as machine-readable knowledge. AIRO [22] models AI risks against the AI Act and ISO standards, the Data Privacy Vocabulary [23] has been extended with an AI Act module, and further efforts represent AI systems, their risks, and their permitted uses as machine-readable documentation and policies [24,25,26,27]. Leveraging these resources would further the interests of actually deploying a RAG system on regulation. We aim to instead measure how the chosen form of knowledge graph affects retrieval and explainability in RAG. The bespoke ontology built here is therefore an experimental control, rather than a proposed standard.

3. Methodology

To answer the main research question, we build two GraphRAG systems over the same section of the EU AI Act: one backed by a Neo4j LPG, and one backed by an RDF ontology derived from the LPG with exactly the same underlying data. Both are evaluated on the same set of questions. The methodology comprises four broad stages, each designed to keep the comparison as fair as possible.

Stage 1: LPG construction. The EU AI Act [1] (first 44 pages, recitals 1-179) is ingested and transformed into a Neo4j LPG using Neo4j's native LLM Graph Builder [7], which automates chunking, entity extraction, relationship extraction, and native vector embedding creation. We use an existing tool rather than a custom implementation as the goal is to represent what an end user would arrive at using standard LPG tools. The first 44 pages are chosen due to the full document's length of 400+ pages; the LLM extraction and query cost would be prohibitive.

Stage 2: LPG analysis. We analyse the created LPG using Python scripts, characterising label and predicate frequencies and their formatting variants. Without understanding the specific patterns and noise in the unconstrained LPG, accurate mapping is impossible.

Stage 3: Ontology design and mapping. A target OWL ontology is designed through domain analysis of the EU AI Act, in tandem with previous stage analysis. Explicit mappings are created to translate the noisy LPG data to this clean ontology, collapsing synonyms and unnecessary granularity into broader classes while preserving the underlying data entirely. These mappings are executed to create the RDF knowledge graph. The RDF RAG system uses Ontotext GraphDB [8] with a native vector index, OWL reasoning, and SPARQL [5].

Stage 4: Evaluation. We use RAGAS [2] rather than ROUGE [17]/BLEU [18]/METEOR [19]. Syntax-overlap metrics fail on legal text (“must” and “must not” both score highly on word overlap). RAGAS uses LLM-as-judge to separately score three metrics: **faithfulness** (is the answer grounded in retrieved context rather than the LLM’s own knowledge?), **context precision** (what proportion of the retrieved content was actually relevant to the question?), and **context recall** (of the information needed to fully answer the question, how much did the retrieval actually find?). These metrics measure retrieval quality (precision and recall) and generation quality (faithfulness) *separately*, which is essential when the two systems share the same LLM and differ only in retrieval. Because the LLM is a constant (GPT-5.2 [9]), any difference in output reflects the graph representation rather than model quirks.

3.1. Neo4j LPG

The unconstrained extraction produced 3,816 entities under 298 labels and 6,552 relationships across 1,861 predicate types: a dense but messy output. The same concept appears under several labels (AI_System, AISystem, AI System), and relationships include synonyms (REGULATES, GOVERNS, LEGISLATES). This inconsistency is expected of schema-less LLM extraction, directly motivating the need for ontology work.

The LPG RAG pipeline uses the native vector index for initial semantic search, then Cypher graph and full-text traversal for context retrieval. It essentially combines three retrieval modes to maximise coverage. An LLM translates natural language into Cypher, retrieves relevant entities, relationships, and chunk text, and passes the context to another agent for answer generation based on curated specific context.

3.2. RDF Ontology and Graph Construction

The RDF ontology was hand-designed through domain analysis, following established ontology engineering principles [10,11].

Mapping decisions were then made through a human-in-the-loop workflow: an agent proposed semantic groupings of entities and predicates, these notions were manually fleshed out into detailed hierarchies, and a second agent validated the output for consistency across all 3,816 entities and 6,552 relationships. The goal of ‘mapping’ is to determine how each LPG entity/relationship should map to the RDF ontology: whether to assign it its own class, merge it into an existing one, or place it as a subclass under a parent.

These decisions required genuine judgement, and in some cases, further domain research. For example, `Right` and `Interest` were kept as separate hierarchy branches due to their distinct legal definitions.

The results are class and property hierarchies with explicit label and predicate mappings, that a Python script (`rdflib` [6]) applies to the LPG data dump to emit the ontology (TBox) and instances (ABox) as 27,340 triples. Entity, chunk, and relationship counts (3,816, 615, 6,552) all match the LPG source exactly. The only removed edges are the 2,672 LPG-internal items (`SIMILARITY`, `NEXT_CHUNK`, `FIRST_CHUNK`), and the vector embeddings are rebuilt natively in RDF. Zero data was lost, meaning the two graphs concretely hold the same data, and any difference in evaluation reflects representation rather than content.

4. Experimental Results

Our first contribution is empirical. Both systems were evaluated on the same 50 questions split into 5 tiers of increasing complexity based on how many distinct pieces of information and graph hops the ground truth answer requires. The tiers are as follows: Simple Factual, Definitional, Multi-hop, Analytical, and Critical. Each question has a manually written reference answer grounded in specific recitals and articles of the EU AI Act [1]. Evaluation uses the RAGAS framework [2].

The LPG leads on all three aggregate metrics, most clearly on faithfulness (0.836 vs 0.595; context precision 0.852 vs 0.818; context recall 0.599 vs 0.524). These overall numbers are misleading. The per-tier breakdown shows the two systems crossing over as questions get harder. The LPG is more faithful than RDF at every tier, scoring a perfect 1.0 on the simple questions and degrading steadily with complexity. This degradation is a retrieval problem, not a generation one: when the LPG retrieves the right context its answer is almost always faithful to it, so what fails on hard questions is finding the context, not using it.

Recall tells the opposite story. On the three hardest tiers the RDF ontology leads (multi-hop 0.63 vs 0.60, analytical 0.76 vs 0.59, critical 0.59 vs 0.41), while the LPG leads on the two easiest (simple 0.85 vs 0.28, definitional 0.54 vs 0.36). These hardest tiers are exactly the questions that need traversal across several parts of the graph, and there the ontology finds more of the information the answer requires. The underlying mechanics of this are covered in Discussion.

The LPG is also erratic where it is weakest (critical-tier recall SD 0.507) against RDF's steadier hard-tier performance (analytical SD 0.179), so RDF's advantage is in both level and consistency. Precision follows the same crossover: the LPG is more precise on simple questions (0.82 vs 0.53), RDF on harder ones (0.96 vs 0.86 on multi-hop). RDF's low precision on simple questions is the cost of its breadth. The same broad retrieval that helps on hard questions pulls in triples a simple question does not need. Structure is not free, and below the complexity threshold it is a net loss. On complex questions that breadth instead returns a higher proportion of relevant content.

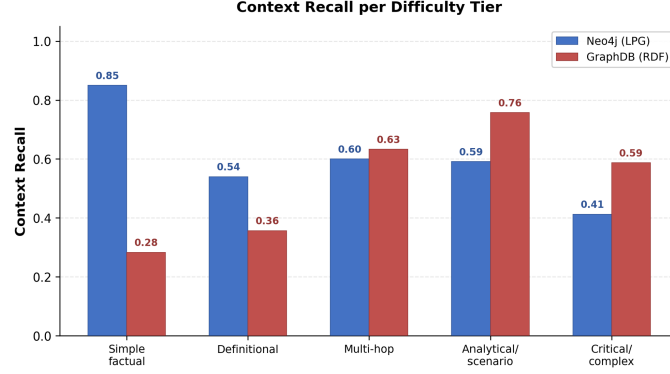


Figure 1. Context recall per difficulty tier. LPG wins on simple questions and the RDF KG wins on harder questions, with a complexity threshold in the middle.

4.1. Discussion

4.1.1. Context Recall Cross-Over

The per-difficulty results show opposite trajectories. The LPG’s retrieval quality degrades as question complexity increases. RDF’s improves. On simple factual questions the LPG wins on every metric. From the multi-hop tier onward RDF achieves higher context recall and higher context precision, finding both more of the needed information and more relevant content. See Figure 1. This crossover is the main finding explained in this section.

4.1.2. Why Structure Helps on Complex Queries

The practical advantage of ontology structure is that it makes retrieval inspectable, and it is easiest to show with an example. In the RDF predicate hierarchy, `regulatoryRelation` is the parent of `regulates`, `prohibits`, `restricts`, and `allows`.

A DESCRIBE query invoking `prohibits` automatically picks up `regulates`, `restricts`, and `allows` as siblings with the same parent. To be exact, that query would return associated parents, types, all associated relationships in both directions (due to inverse predicates), plus the immediate hierarchy surrounding it. The LLM receiving this context can see not just what the entity/relation is, but where it sits in the ontology and what it connects to, which makes the retrieval path auditable.

In Cypher, relationships are just labels, and there is no ontology. `REGULATES` and `PROHIBITS` are unrelated strings unless the query explicitly lists both. If the LLM generating the query writes `[:REGULATES]`, only `[:REGULATES]` is returned, whether or not other semantically associated predicates exist. Same with entity types: a SPARQL query invoking `Actor` implicitly covers `Organisation`, `GovernmentBody`, `Person`, `Stakeholder`, and `MarketActor` as related children. Schema-less LPG/Cypher cannot do this. The RDF structure widens retrieval coverage at each step, catching relevant items that a schema-less system misses.

4.1.3. Worked Example: Question 42

Question 42 asks: “Define what constitutes a ‘serious incident’, connect it to post-market monitoring obligations, and trace the governance chain: who monitors, who receives reports, and how does this differ for AI systems built on general-purpose AI models?”

This is a complex analytical question. The ground truth lists 4 pieces of information spanning Recitals 153, 155, and 161.

The RDF pipeline used full-text search to land on the `serious_incident` entity, then DESCRIBE to pull in everything connected to it in both directions: its definition, the causal chain (use of a GPAI model causes a serious incident), the responsible actor, and its place in the class hierarchy (`Incident subclassOf Event`).

Two further queries retrieved the remaining context needed. RDF scored 1.0 on recall and 1.0 on faithfulness. The LPG scored 0.0 on both. Its vector search retrieved content adjacent to “monitoring” and “governance” but structurally unrelated to serious incidents, because semantic similarity is not the same as a structural connection.

4.2. What is Being Compared: Paradigms or Retrieval Strategies?

The two systems differ in two ways at once: in how they represent knowledge (a schema-less property graph against a formal ontology) and in how they retrieve it (vector plus Cypher against full-text plus DESCRIBE). This raises a fair question: are we comparing knowledge-graph paradigms, or just two retrieval strategies? We do not try to equalise the pipelines, because artificially matched retrieval strategies would test neither system as it would actually be deployed. Instead we compare each paradigm in its native form. The mechanism (Section 4.1.2) demonstrates something that Cypher cannot reproduce regardless of retrieval tactic; even the LPG’s reliance on vector localisation is a consequence of having no schema to leverage. The comparison is therefore between paradigms as deployed, not a claim that representation is the only meaningful difference.

4.3. LPG Failure Modes

The LPG’s failures on hard questions are specific and repeatable, and both stem from its dependence on initial vector localisation. Vector localisation works when the query maps to one close chunk (e.g. ‘what does Article 5 prohibit?’). However, on Q42 the words “monitoring” and “governance” are semantically close to many irrelevant chunks, so vector search starts from the wrong entity and Cypher then traverses correctly from the wrong place, with no way to recover. Separately, the Act’s long articles are split across chunks, so the chunk holding the query terms and the chunk holding the answer are often different: vector search finds the former and stops. RDF avoids both problems because it does not depend on a semantic guess to find its starting point. Its full-text search lands on the matched entity and the ontology then pulls in everything structurally connected to it, regardless of which chunk the text came from. The advantage therefore stems from the ontology’s structure replacing the LPG RAG system’s reliance on semantic similarity, leaving the LPG behaving like the flat vector RAG a graph is meant to improve on.

4.4. Retrieval/Utilisation Gap

In 10 of the 30 questions where RDF retrieved adequate context (recall ≥ 0.5), the answer still scored below 0.5 on faithfulness. The right information was retrieved but did not

survive into the answer: in governance terms, a provenance failure. The cause is volume. RDF's DESCRIBE returns full subgraphs averaging 5,544 characters against the LPG's 2,097, padded with namespace declarations, sibling classes and full URIs, so the answer is present but buried in noise the LLM does not extract.

Even where the ontology-afforded structure wins on retrieval, that high recall does not guarantee faithfulness. The same content is present in LPG and RDF returned contexts, but the effort needed to extract it is very different. Ultimately, it is a presentation issue that can be fixed without making major architectural changes.

4.5. Limitations

The study uses 50 questions on 44 pages, which is a limited sample. The RDF graph is derived from the LPG, so extraction errors carry over. The RAGAS framework is LLM-as-judge, which introduces its own bias, and risks sycophancy. The transformation pipeline is reusable in principle, but its custom and labour-intensive nature limits the practicability of replication in a new domain. The ontology is fit for this experiment, not for reuse; aligning it to shared vocabularies such as DPV would make it interoperable and is the natural next step. It is released as an experimental artefact for reproducibility, not as a proposed AI Act vocabulary. Lastly, using the full AI Act plus other sources like the GDPR [20] and the NIST AI RMF [21] would better stress-test both pipelines, and is where RDF's global URIs should give the most structural advantage.

5. Conclusion

Empirically, LPG and RDF have complementary retrieval strengths that cross over between simple and complex questions. The ontology's class and predicate hierarchies widen retrieval and recover more context on hard questions, but this is not a uniform good. The breadth afforded by ontology structure is dead weight on simple ones, degrading precision. Meanwhile, despite being a graph, the LPG inherits flat vector RAG's central weakness of reliance on vector similarity as a starting point, degrading recall on harder questions.

These findings point to a logical solution to the problem: a single RDF system with complexity-dependent post-processing, stripping DESCRIBE noise for simple questions while keeping full context for complex ones. This works because the ontology's weakness on simple questions is one of presentation rather than retrieval. In 10 of 30 questions where RDF's recall was adequate but faithfulness was not, the needed content was already retrieved albeit lost in context noise. This also matches design intuitions in existing work: PolyG's [14] assertion that no single retrieval strategy serves both simple and complex questions, and the use of re-ranking to filter broad retrieval [15]. In a compliance setting the hard questions are the ones that matter, and on those the ontology retrieves more of what is needed. More importantly, the real advantage is making retrieval inspectable and auditable in a way that the LPG is not. When mistakes carry legal consequences, structure earns its cost.

References

- [1] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 (Artificial Intelligence Act). *Official Journal of the European Union*, L 2024/1689, 2024.
- [2] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In *Proc. 18th Conf. EACL: System Demonstrations*, pp. 150–158, 2024.
- [3] R. Cyganiak, D. Wood, and M. Lanthaler. RDF 1.1 Concepts and Abstract Syntax. W3C Recommendation, 2014. <https://www.w3.org/TR/rdf11-concepts/>
- [4] W3C OWL Working Group. OWL 2 Web Ontology Language Document Overview (Second Edition). W3C Recommendation, 2012. <https://www.w3.org/TR/owl2-overview/>
- [5] S. Harris and A. Seaborne. SPARQL 1.1 Query Language. W3C Recommendation, 2013. <https://www.w3.org/TR/sparql11-query/>
- [6] RDFLib Team. RDFLib: A Python library for working with RDF. 2024. <https://github.com/RDFLib/rdfLib>
- [7] Neo4j, Inc. Neo4j LLM Graph Builder Documentation. 2024. <https://neo4j.com/labs/genai-ecosystem/llm-graph-builder/>
- [8] Ontotext. GraphDB Free Edition, 2024. <https://www.ontotext.com/products/graphdb/>
- [9] OpenAI. Introducing GPT-5.2. 2025. <https://openai.com/index/introducing-gpt-5-2/>
- [10] T. R. Gruber. A Translation Approach to Portable Ontology Specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.
- [11] N. F. Noy and D. L. McGuinness. Ontology Development 101: A Guide to Creating Your First Ontology. Stanford KSL Tech. Report KSL-01-05, 2001.
- [12] B. Menaouer, C. Younes, E. E. Mokhtar, et al. Pharmaceutical Chatbot Assistant Using Generative AI and Knowledge Graph with Specific Pharmaceutical Resources. *Iran Journal of Computer Science*, 9, article 40, 2026. <https://doi.org/10.1007/s42044-026-00396-3>
- [13] A. Grover and J. Leskovec. node2vec: Scalable Feature Learning for Networks. In *Proc. ACM SIGKDD (KDD)*, pp. 855–864, 2016.
- [14] R. Liu, H. Jiang, X. Yan, B. Tang, and J. Li. PolyG: Adaptive Graph Traversal for Diverse GraphRAG Questions. arXiv:2504.02112, 2025.
- [15] K. Guo, X. Dai, S. Zeng, et al. Beyond Static Retrieval: Opportunities and Pitfalls of Iterative Retrieval in GraphRAG. arXiv:2509.25530, 2025.
- [16] P. Patel. Graph-Enhanced Retrieval-Augmented Question Answering for E-Commerce Customer Support. arXiv:2509.14267, 2025.
- [17] C.-Y. Lin. ROUGE: A Package for Automatic Evaluation of Summaries. In *Text Summarization Branches Out*, pp. 74–81, 2004.
- [18] K. Papineni, S. Roukos, T. Ward, et al. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proc. 40th Annual Meeting of the ACL*, pp. 311–318, 2002.
- [19] S. Banerjee and A. Lavie. METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments. In *Proc. ACL Workshop on Eval. Measures for MT and Summarization*, pp. 65–72, 2005.
- [20] European Parliament and Council of the European Union. Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union*, L 119, 2016, pp. 1–88.
- [21] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, 2023. <https://doi.org/10.6028/NIST.AI.100-1>
- [22] D. Golpayegani, H. J. Pandit, and D. Lewis. AIRO: An Ontology for Representing AI Risks Based on the Proposed EU AI Act and ISO Risk Management Standards. In *Towards a Knowledge-Aware AI (SEMANTICS 2022)*, vol. 55, pp. 51–65, IOS Press, 2022.
- [23] H. J. Pandit, B. Esteves, G. P. Krog, et al. Data Privacy Vocabulary (DPV) Version 2, EU AI Act Extension (EU-AIAct). W3C DPVCG, 2024. <https://w3c.github.io/dpv/2.2/legal/eu/aiact/>
- [24] D. Golpayegani, I. Hupont, C. Panigutti, et al. AI Cards: Towards an Applied Framework for Machine-Readable AI and Risk Documentation Inspired by the EU AI Act. In *Privacy Technologies and Policy (APF 2024)*, pp. 48–72, Springer, 2024.
- [25] D. Golpayegani, H. J. Pandit, and D. Lewis. AICat: An AI Cataloguing Approach to Support the EU AI Act. arXiv:2501.04014, 2025.
- [26] D. Golpayegani, B. Esteves, H. J. Pandit, and D. Lewis. AIUP: An ODRL Profile for Expressing AI Use Policies to Support the EU AI Act. CEUR Workshop Proc., Vol. 3759, paper 17, 2024.
- [27] D. Golpayegani, H. J. Pandit, and D. Lewis. Semantic Patterns of Prohibited AI Systems in the EU AI Act. In *SEMANTICS 2025 (Posters, Demos, Workshops & Tutorials)*, CEUR Workshop, Vol. 4064, 2025.